SkillMatch

Team 6: Elias Woldie, Toan Tran, Ivy Ly, Hamzah Qutob, Eren Yolcuoglu

Table of Contents

Introduction	01
User Requirements	02
System Requirements	03
Use Cases	04
Architecture	05
Implementation	06
Testing Strategy	07
Running System Demo	08

Introduction

Project Description

- SkillMatch is a dynamic web application that allows skilled service providers referred to as “workers” to match with potential service seekers referred to as “employers” to connect, meet, and provide/receive a service based on their location.

Design and Implementation

- The Frontend of SkillMatch is developed using **React**, it allows us to create an engaging, responsive and dynamic experience for the users. We utilize Django as it is used as the core backend which provides a framework for development. Then we used...

User Requirements

Requirements Received:

- Both Users (Workers and Employers):
  - Create user profile
  - Connect through a contract
- Worker:
  - View job postings
  - Apply for job
  - Update service status (completed or not)
- Employer:
  - Post Job requests
  - Review/ rate worker's service

Functional Requirements:

- Account creation and user authentication
- Receiving notification when there is a new job request

Non-Functional Requirements:

- Application is easy to navigate for all users
- Application is efficient, load content and respond to user inputs without delay

Requirements Changed: Swapped Employer and Worker requirements

System Requirements

Requirements Received:

- Ability for users to register and create profiles that reflects their role (worker or employer)
- Enable the employer to post a job request.
- System utilizing location data to show the workers who are nearby
- Application is easy to use

Requirements Changed:

- Show the workers based on location for now

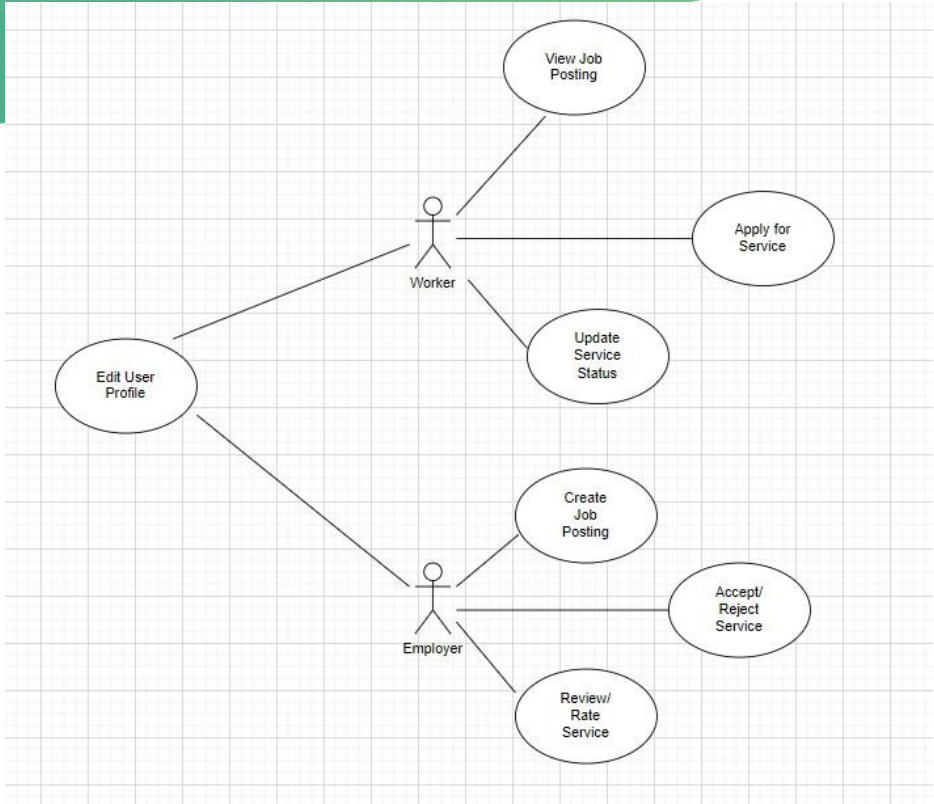
Functional Requirements:

- Enable users to create and manage their profiles
- System shows the workers that are nearby based on location proximity
- Employers can request a job from the workers.
- Review system. Employers are able to leave a rating

Non-Functional Requirements:

- Ensure the application is intuitive and user-friendly
- Achieve the optimal performance under the expected load

Use Case Diagram



Use Case 1: Create Job Posting

Identifier	Use Case #1: Create Job Posting
Purpose	Allow Employers to create job request to find potential workers to complete a service.
Requirements	As a service-seeking user, I want to be able to create job postings to find potential workers to complete a task.
Development Risks	None
Pre-Conditions	Employer is on the job request page.
Post-Conditions	Job posting is visible for the potential workers.

Use Case: Create Job Posting (Typical Course of Action)

Seq#	Actor's Action	System's Response
1	Employer navigates to the job request page	
2	Employer fills out all required input fields	
3	Employer selects "Post Job" button	
4		The system will save job posting information in the database
5		The system displays the job's information on worker's dashboard page

Use Case: Create Job Posting (Alternate Course of Action)

Seq#	Actor's Action	System's Response
1	Employer navigates to the job request page	
2	Employer fills out some of the required input fields	
3	Employer selects "Post Job" button	
4		The system will not save job posting information in the database
5	Employer fills out the rest of the required input fields	
6	Employer selects "Post Job" button	
7		The system will save job posting information in the database and displays it on the worker's dashboard

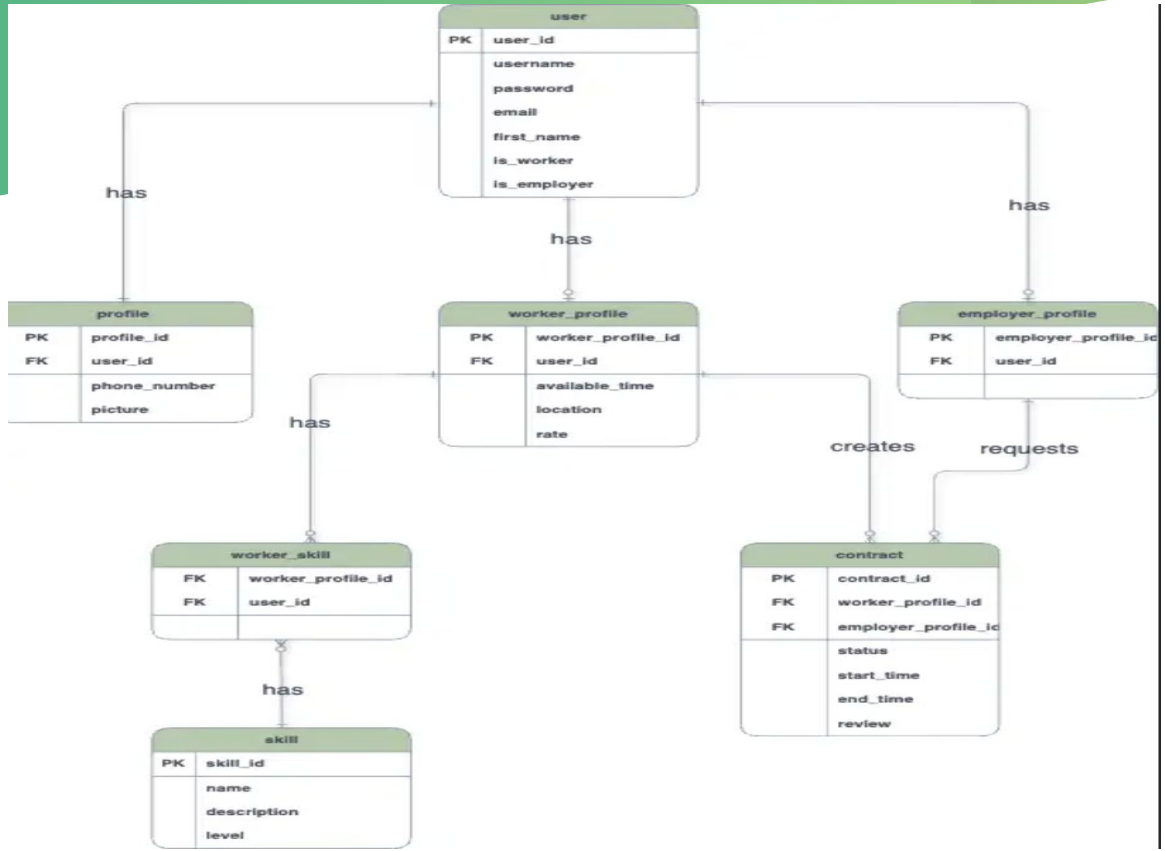
Use Case: Create Job Posting (Exceptional Course of Action)

Seq#	Actor's Action	System's Response
1	Employer navigates to the job request page	
2	Employer doesn't fill out all required input fields	
3	Employer selects "Post Job" button	
4		The system will not save job posting information in the database
5		The system doesn't displays the job's information on worker's dashboard page

Architecture

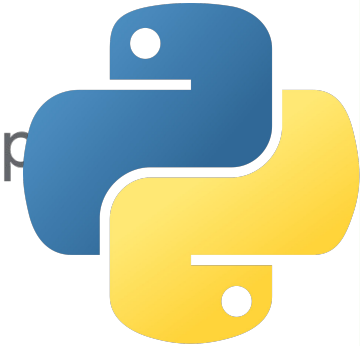
- Backend Architecture:
  - Django handles logic and data management.
  - REST Framework for building scalable APIs.
- Frontend Architecture:
  - React components for modular and dynamic UI.
  - Bootstrap for responsive styling.
- Database Architecture:
  - PostgreSQL database with tables for users, services,

ER Diagram


user
PK user_id
username
password
email
first_name
is_worker
is_employer
has
has
has
profile
PK profile_id
FK user_id
phone_number
picture
worker_profile
PK worker_profile_id
FK user_id
available_time
location
rate
employer_profile
PK employer_profile_id
FK user_id
has
creates
requests
worker_skill
FK worker_profile_id
FK user_id
contract
PK contract_id
FK worker_profile_id
FK employer_profile_id
status
start_time
end_time
review
has
skill
PK skill_id
name
description
level

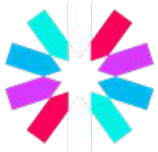
Tech Stack

Map



GitHub

PostgreSQL



JWT

Implementation

Process

- Frontend: HTML, CSS, JSX
- Backend: Python
- IDE: Visual Studio Code

Software

- Tools: Git, Github,
- Framework: React, Django

Deployment

- Locally hosted

Testing Strategy

What we tested

- API Endpoints
- HTTP Request
- Networks

POSTMAN

Demo